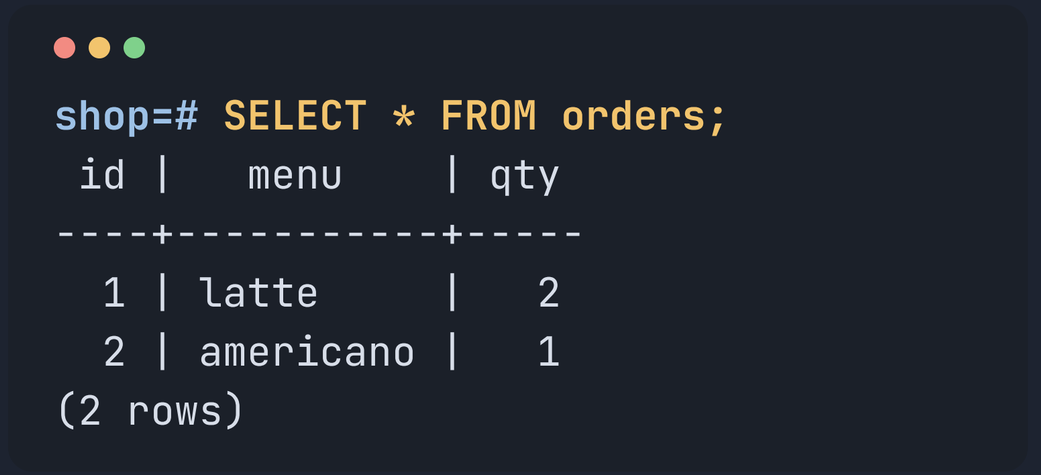
KANT AX/AI 엔지니어 트랙 · 34

Docker 4

PostgreSQL 컨테이너로 데이터베이스 쓰기

33의 MySQL과 같은 일을 PostgreSQL로 해요



오늘의 순서

1	**PostgreSQL 컨테이너 켜기** pull, run, -e, logs, psql로 들어가기	5–9쪽
2	**데이터베이스와 표 만들기** CREATE DATABASE, \c, \l, CREATE TABLE, \dt, \d, INSERT, SELECT, DROP	10–15쪽
3	**계정과 접속 방법** postgres와 일반 계정, POSTGRES_USER, bash로, 밖에서 비밀번호, -c 한 문장	16–20쪽
4	**볼륨과 compose** 지우면 사라지는 표, /var/lib/postgresql, compose.yaml	21–25쪽
5	**자주 나는 오류** 비밀번호 빠짐, 너무 빨리 접속, 5432 겹침, 세미콜론, 18의 볼륨 자리, 남은 볼륨	26–31쪽
6	**실제 상황 예제** 다음 날에도 남는 주문, init 폴더로 메뉴 채우기	32–33쪽

3–4쪽은 들어가기, 34–37쪽은 MySQL과 비교, 막힐 때, 한 장 요약이에요.

들어가기

MySQL 명령을 그대로 치면 PostgreSQL에서는 안 돼요

```
~ $ docker run -d --name pg -e POSTGRES_PASSWORD=cafe1234 -p 5432:5432 postgres:18
b0771208641d…

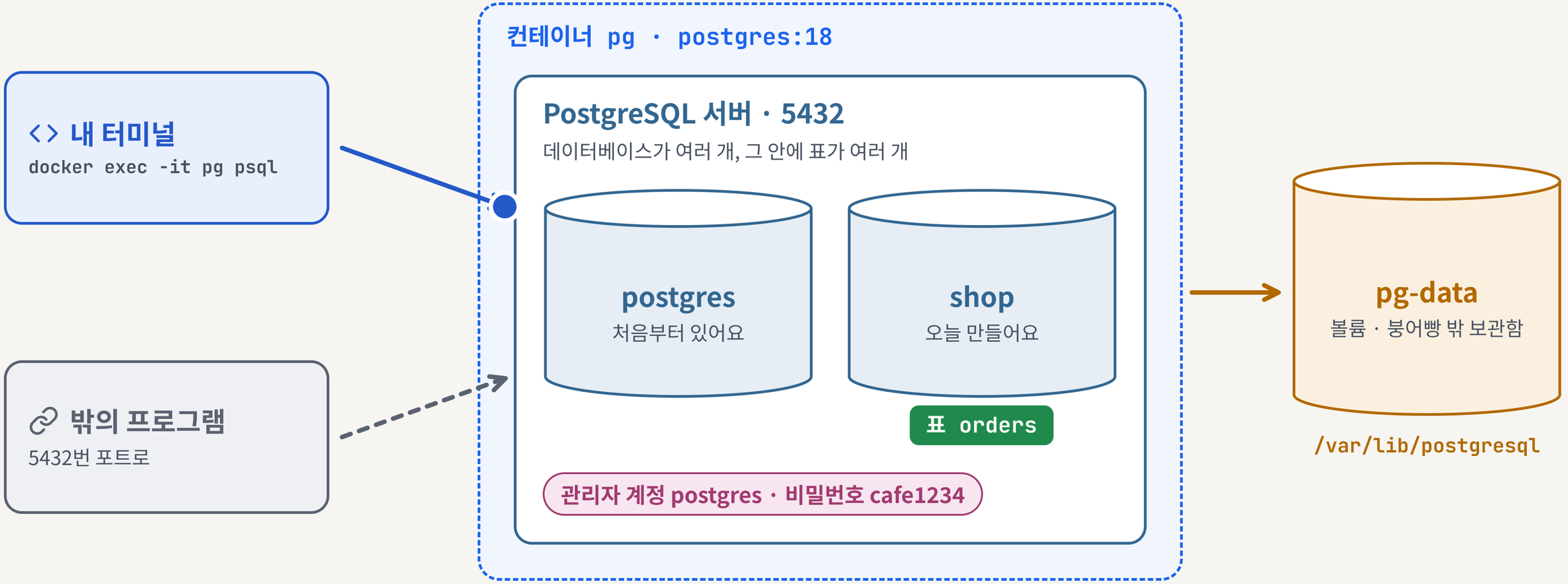
~ $ docker exec -it pg mysql -u root -p
OCI runtime exec failed: exec failed: unable to start container process: exec: "mysql": executable file not found in $PATH
```

같은 일, 다른 이름	33 MySQL	34 PostgreSQL
이미지	`mysql:8.4`	`postgres:18`
들어가는 프로그램	`mysql -u root -p`	`psql -U postgres`
포트	`3306`	`5432`

켜고, 표를 만들고, 볼륨으로 남기는 순서는 33과 같아요. 이름과 명령만 바꿔 익혀요.

들어가기

pg 컨테이너 안에서 PostgreSQL 서버가 5432번으로 기다려요



1. PostgreSQL 컨테이너 켜기

Docker Hub의 공식 이미지 postgres:18을 받아요

```
~ $ docker pull postgres:18
18: Pulling from library/postgres
37fb86c2a767: Pull complete
fcd0e9514062: Pull complete
6d141360b363: Pull complete
a28de24f57c0: Pull complete
e9dd37612d7b: Pull complete
369949eb0229: Pull complete
7e0f86eb8508: Pull complete
da434b32d480: Pull complete
bcc75ab4e010: Pull complete
eac25292b613: Pull complete
fc6c21aca07c: Pull complete
cbe0afc9eb7b: Pull complete
2a59118cca0e: Pull complete
2f3707deb487: Download complete
a82d2da6710e: Download complete
Digest: sha256:e3d24f6906d0760bf4da0dae872415391edededefad23fd471c47dc53b0ddf3c42
Status: Downloaded newer image for postgres:18
docker.io/library/postgres:18

~ $ docker images postgres
IMAGE          ID             DISK USAGE   CONTENT SIZE   EXTRA
postgres:18    e3d24f6906d0       650MB          169MB
```

다 받았어요

postgres

Docker Official Image

PostgreSQL 팀과 Docker가 함께 관리해요

태그 고르기

18 17 latest

18은 PostgreSQL 18이에요. latest는 새 버전이 나오면 바뀌니 숫자로 적어요.

받은 이미지는 650MB예요. 처음 받을 때만 기다려요.

1. PostgreSQL 컨테이너 켜기

docker run 한 줄로 PostgreSQL 서버를 켜요

```
docker run -d --name pg -e POSTGRES_PASSWORD=cafe1234 -p 5432:5432 postgres:18
```

- `-d` : 뒤에서 켜기
- `--name pg` : 이름 pg
- `-e POSTGRES_PASSWORD=cafe1234` : 관리자 비밀번호 (꼭 있어야 해요)
- `-p 5432:5432` : 5432번 문 열기
- `postgres:18` : 이미지

```
~ $ docker run -d --name pg -e POSTGRES_PASSWORD=cafe1234 -p 5432:5432 postgres:18
b0771208641d3ea65dd6d5fc40f00ff656f22a770051d86555e155230e78155c

~ $ docker ps
CONTAINER ID   IMAGE         COMMAND                  CREATED         STATUS         PORTS                    NAMES
b0771208641d   postgres:18   "docker-entrypoint.s…"   3 seconds ago   Up 2 seconds   0.0.0.0:5432->5432/tcp   pg
```

새로 나온 건 -e 하나예요. 나머지는 29의 docker run 그대로예요.

1. PostgreSQL 컨테이너 켜기

-e로 건네는 이름 세 개가 계정과 데이터베이스를 정해요

-e 이름	정하는 것	오늘 값
POSTGRES_PASSWORD	**관리자 비밀번호** 꼭 적어야 해요	cafe1234
POSTGRES_USER	**관리자 이름** 안 적으면 postgres	postgres
POSTGRES_DB	**처음 만들 데이터베이스** 안 적으면 관리자 이름과 같아요	postgres

```
~ $ docker exec pg env
HOSTNAME=b0771208641d
POSTGRES_PASSWORD=cafe1234
GOSU_VERSION=1.19
LANG=en_US.utf8
PG_MAJOR=18
PG_VERSION=18.6-1.pgdg13+2
PGDATA=/var/lib/postgresql/18/docker
HOME=/root
```

오늘은 비밀번호만 건네요. 관리자는 postgres, 데이터베이스도 postgres로 시작해요.

POSTGRES_USER와 POSTGRES_DB는 3파트와 4파트에서 직접 써 봐요.

1. PostgreSQL 컨테이너 켜기

logs에 ready to accept connections가 두 번 나오면 준비 끝이에요

```
~ $ docker logs pg
The files belonging to this database system will be owned by user "postgres".
…
… LOG:  database system is ready to accept connections
server started
…
server stopped
PostgreSQL init process complete; ready for start up.
…
… LOG:  listening on IPv4 address "0.0.0.0", port 5432
… LOG:  database system is ready to accept connections
```

준비용 서버 (밖에서 못 들어와요)

준비 끝, 진짜 서버를 켜요

5432번에서 기다려요

처음 켤 때는 몇 초 걸려요. 마지막 줄이 보이면 들어가요.

1. PostgreSQL 컨테이너 켜기

docker exec -it로 컨테이너 안의 psql을 열어요

```
docker exec -it pg psql -U postgres
```

- `exec -it` — 30에서 배운 컨테이너 안에서 실행
- `pg` — PostgreSQL에 접속하는 프로그램
- `psql -U postgres` — 계정 이름 (User)

```
~ $ docker exec -it pg psql -U postgres
psql (18.6 (Debian 18.6-1.pgdg13+2))
Type "help" for help.

postgres=# \q
~ $
```

postgres=#

앞은 지금 들어와 있는 데이터베이스 이름이에요. 이 뒤에 SQL을 쳐요.

\q

psql을 나와 내 터미널로 돌아와요. 컨테이너는 계속 돌아요.

2. 데이터베이스와 표 만들기

psql에는 SQL과 백슬래시 명령, 두 가지를 쳐요

SQL

데이터베이스에 시키는 말이에요. 33 MySQL에서도 같아요. 끝에 ;를 붙여야 실행돼요.

```
CREATE DATABASE shop;
```

```
CREATE TABLE orders (…);
```

```
INSERT INTO orders VALUES (…);
```

```
SELECT * FROM orders;
```

```
DROP TABLE orders;
```

백슬래시 명령

psql이 알아듣는 짧은 명령이에요. \로 시작하고 ;를 안 붙여요.

`\l`	데이터베이스 목록
`\c shop`	shop으로 이동
`\dt`	표 목록
`\d orders`	표 구조
`\du`	계정 목록
`\q`	나가기

CREATE DATABASE로 만들고 \l로 목록을 봐요

```
~ $ docker exec -it pg psql -U postgres
postgres=# CREATE DATABASE shop;
CREATE DATABASE
postgres=# \l
                                                 List of databases
   Name    |  Owner   | Encoding | Locale Provider |  Collate   |   Ctype    | Locale | ICU Rules |   Access privileges
-----------+----------+----------+-----------------+------------+------------+--------+-----------+-----------------------
 postgres  | postgres | UTF8     | libc            | en_US.utf8 | en_US.utf8 |        |           |
 shop      | postgres | UTF8     | libc            | en_US.utf8 | en_US.utf8 |        |           |
 template0 | postgres | UTF8     | libc            | en_US.utf8 | en_US.utf8 |        |           | =c/postgres          +
           |          |          |                 |            |            |        |           | postgres=CTc/postgres
 template1 | postgres | UTF8     | libc            | en_US.utf8 | en_US.utf8 |        |           | =c/postgres          +
           |          |          |                 |            |            |        |           | postgres=CTc/postgres
(4 rows)

postgres=# \q
```

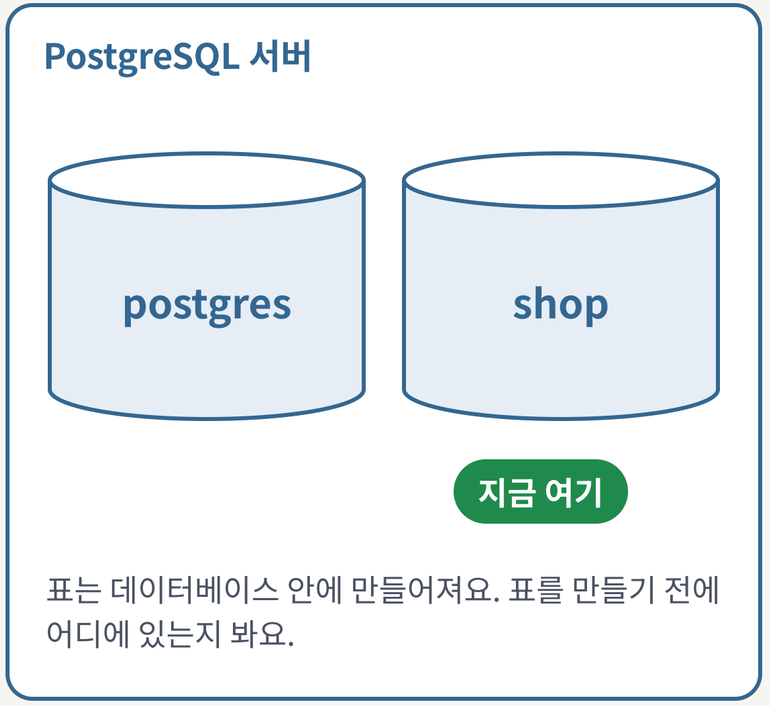
다 만들었다는 답이에요

데이터베이스 목록

postgres는 처음부터 있던 것, template0과 template1은 PostgreSQL이 새 데이터베이스를 만들 때 쓰는 틀이에요.

\c shop으로 옮기면 앞의 이름이 shop=#로 바뀌어요

```
~ $ docker exec -it pg psql -U postgres
postgres=# \c shop
You are now connected to database "shop" as user "postgres".
shop=# \q
```



shop=#

psql을 새로 열 때 -d shop을 붙이면 처음부터 shop으로 들어가요.

```
docker exec -it pg psql -U postgres -d shop
```

CREATE TABLE로 표를 만들고 \dt, \d로 확인해요

```
~ $ docker exec -it pg psql -U postgres -d shop
shop=# CREATE TABLE orders (id INT PRIMARY KEY, menu VARCHAR(20), qty INT);
CREATE TABLE
shop=# \dt
          List of tables
 Schema |  Name  | Type  |  Owner
--------+--------+-------+----------
 public | orders | table | postgres
(1 row)

shop=# \d orders
                    Table "public.orders"
 Column |         Type          | Collation | Nullable | Default
--------+-----------------------+-----------+----------+---------
 id     | integer               |           | not null |
 menu   | character varying(20) |           |          |
 qty    | integer               |           |          |
Indexes:
    "orders_pkey" PRIMARY KEY, btree (id)
```

표 목록

orders 표의 열

INT

VARCHAR(20)

id가 기본 키예요

INT는 integer, VARCHAR는 character varying으로, 긴 이름으로 보여 줘요.

INSERT로 행을 넣고 SELECT로 꺼내 봐요

```
~ $ docker exec -it pg psql -U postgres -d shop
shop=# INSERT INTO orders VALUES (1, 'latte', 2);
INSERT 0 1
shop=# INSERT INTO orders VALUES (2, 'americano', 1);
INSERT 0 1
shop=# SELECT * FROM orders;
 id |   menu    | qty
----+-----------+-----
  1 | latte     |   2
  2 | americano |   1
(2 rows)

shop=# SELECT menu, qty FROM orders;
   menu    | qty
-----------+-----
 latte     |   2
 americano |   1
(2 rows)
```

1행 넣었어요

orders

id INT	menu VARCHAR(20)	qty INT
1	latte	2
2	americano	1

값은 표를 만들 때 적은 열 순서대로 넣어요. 글자는 '작은따옴표'로 감싸요.

*는 모든 열이에요. 열 이름을 적으면 그 열만 나와요.

DROP으로 지워요. 들어가 있는 데이터베이스는 못 지워요

```
~ $ docker exec -it pg psql -U postgres -d shop
shop=# DROP TABLE orders;
DROP TABLE
shop=# \dt
Did not find any tables.
shop=# DROP DATABASE shop;
ERROR:  cannot drop the currently open database
shop=# \c postgres
You are now connected to database "postgres" as user "postgres".
postgres=# DROP DATABASE shop;
DROP DATABASE
```

표가 없어요

cannot drop the currently open database

지금 들어가 있는 데이터베이스라서 못 지운다는 뜻이에요.

\c postgres로 다른 데이터베이스에 옮긴 뒤 지워요.

> DROP은 되돌릴 수 없어요. 안의 표와 행도 같이 지워져요. 연습용에서만 쳐요.

3. 계정과 접속 방법

관리자는 =#, 일반 계정은 =>로 표시가 달라요

```
~ $ docker exec -it pg psql -U postgres
postgres=# CREATE USER app WITH PASSWORD 'app1234';
CREATE ROLE
postgres=# \du
                             List of roles
 Role name |                         Attributes
-----------+------------------------------------------------------------
 app       |                                                            
 postgres  | Superuser, Create role, Create DB, Replication, Bypass RLS
```

권한 없음

관리자 (Superuser)

```
~ $ docker exec -it pg psql -U app -d postgres
postgres=> CREATE DATABASE test;
ERROR:  permission denied to create database
```

만들 권한이 없어요

앱이 쓸 계정은 관리자 대신 일반 계정으로 만들어요. 실수로 데이터베이스를 지우지 못해요.

3. 계정과 접속 방법

POSTGRES_USER를 주면 그 이름이 관리자가 되고 postgres는 없어요

```
~ $ docker run -d --name pg2 -e POSTGRES_USER=cafe -e POSTGRES_PASSWORD=cafe1234 -p 5433:5432 postgres:18
4087ba29a61a…

~ $ docker exec -it pg2 psql -U postgres
psql: error: connection to server on socket "/var/run/postgresql/.s.PGSQL.5432" failed: FATAL:  role "postgres" does not exist

~ $ docker exec -it pg2 psql -U cafe
cafe=# \du
                             List of roles
 Role name |                         Attributes
-----------+------------------------------------------------------------
 cafe      | Superuser, Create role, Create DB, Replication, Bypass RLS

cafe=# CREATE DATABASE test;
CREATE DATABASE
```

cafe가 관리자

관리자라서 돼요

33 MySQL의 MYSQL_USER

일반 계정을 하나 더 만들어요. root는 그대로 있어요.

34 PostgreSQL의 POSTGRES_USER

관리자 이름을 바꿔요. postgres 계정은 안 생겨요.

3. 계정과 접속 방법

bash로 들어가서 psql을 쳐도 돼요. 안에서는 비밀번호를 안 물어요

```
~ $ docker exec -it pg bash
root@b0771208641d:/# psql -U postgres
postgres=# \q
root@b0771208641d:/# psql -U postgres -h localhost
postgres=# \conninfo
      Connection Information
      Parameter       |   Value
----------------------+-----------
 Host                 | localhost
 Password Used        | false
…
postgres=# \q
root@b0771208641d:/# exit
exit
```

비밀번호 안 물어요

-h localhost도 그대로

컨테이너 안에서 오는 접속은 믿고 들여보내요.

공식 이미지가 그렇게 설정해 둬요. 이걸 trust라고 불러요.

root@…:/#는 컨테이너 안 bash예요.

psql을 나와도 아직 컨테이너 안이에요. exit로 내 터미널에 돌아와요.

3. 계정과 접속 방법

밖에서 5432번으로 붙으면 비밀번호를 물어요

```
docker run --rm -it --add-host host.docker.internal:host-gateway postgres:18 psql -h host.docker.internal -U postgres
```

- `docker run --rm -it` — 잠깐 쓰고 지울 컨테이너 하나 더
- `--add-host host.docker.internal:host-gateway` — 이 이름을 쓰게 해 줘요 (Mac, Windows는 없어도 돼요)
- `postgres:18` — psql이 들어 있는 이미지
- `psql -h host.docker.internal -U postgres` — 내 컴퓨터 5432번으로 접속

```
~ $ docker run … psql -h host.docker.internal -U postgres
Password for user postgres:
postgres=# \conninfo
           Connection Information
      Parameter       |        Value
----------------------+----------------------
 Host                 | host.docker.internal
 Server Port          | 5432
 Password Used        | true
…
```

cafe1234를 쳤어요 (화면엔 안 보여요)

```
~ $ docker run … -U postgres
Password for user postgres:
psql: error: connection to server at
"host.docker.internal" (172.17.0.1), port 5432
failed: FATAL:  password authentication failed for
user "postgres"
```

cafe123

나중에 파이썬 앱도 이렇게 밖에서 5432번으로 붙어요. 그때 비밀번호가 필요해요.

3. 계정과 접속 방법

-c를 붙이면 SQL 한 문장만 실행하고 바로 나와요

```
docker exec pg psql -U postgres -c "SELECT version();"
```

-it 없이

이 문장 하나만 실행

```
~ $ docker exec pg psql -U postgres -c "SELECT version();"
                                                 version
-------------------------------------------------------------------------------------------------------------
 PostgreSQL 18.6 (Debian 18.6-1.pgdg13+2) on x86_64-pc-linux-gnu, compiled by gcc (Debian 14.2.0-19) 14.2.0, 64-bit
(1 row)

~ $ docker exec pg psql -U postgres -c "\l"
                                                  List of databases
   Name    |  Owner   | Encoding | Locale Provider |  Collate   |   Ctype    | Locale | ICU Rules |   Access privileges
-----------+----------+----------+-----------------+------------+------------+--------+-----------+-----------------------
 postgres  | postgres | UTF8     | libc            | en_US.utf8 | en_US.utf8 |        |           |
 template0 | postgres | UTF8     | libc            | en_US.utf8 | en_US.utf8 |        |           | =c/postgres          +
           |          |          |                 |            |            |        |           | postgres=CTc/postgres
 template1 | postgres | UTF8     | libc            | en_US.utf8 | en_US.utf8 |        |           | =c/postgres          +
           |          |          |                 |            |            |        |           | postgres=CTc/postgres
(3 rows)
```

-it가 없어서 psql 화면에 들어가지 않아요. 결과만 찍고 내 터미널로 돌아와요.

4. 볼륨과 compose

볼륨 없이 켠 컨테이너를 지우면 데이터베이스도 같이 사라져요

```
~ $ docker exec -it pg psql -U postgres
postgres=# CREATE DATABASE shop;
CREATE DATABASE
postgres=# \c shop
You are now connected to database "shop" as user "postgres".
shop=# CREATE TABLE orders (…);
CREATE TABLE
shop=# INSERT INTO orders VALUES (1, 'latte', 2);
INSERT 0 1
shop=# INSERT INTO orders VALUES (2, 'americano', 1);
INSERT 0 1
```

```
~ $ docker stop pg
pg
~ $ docker rm pg
pg
~ $ docker run -d --name pg … postgres:18
02bc8151523b…

~ $ docker exec -it pg psql -U postgres -d shop
psql: error: connection to server on socket
"/var/run/postgresql/.s.PGSQL.5432" failed: FATAL:  database "shop"
does not exist
```



컨테이너를 지우면 그 안의 데이터 폴더도 같이 지워져요.

새로 켠 pg에는 처음부터 있던 postgres 데이터베이스만 있어요.

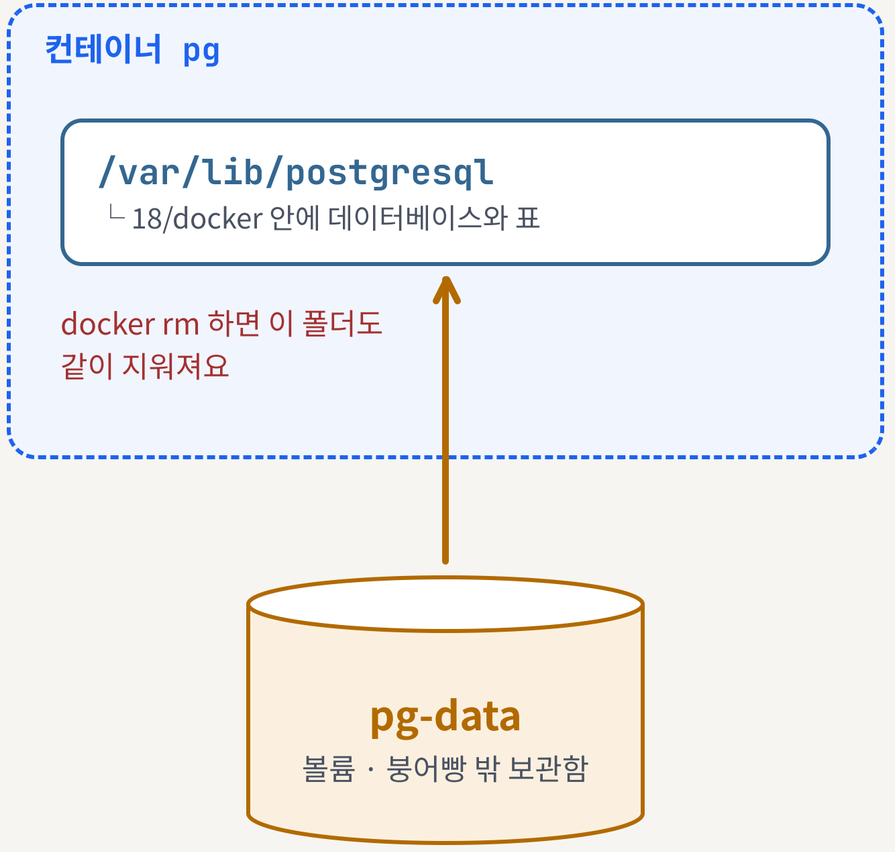
4. 볼륨과 compose

데이터는 컨테이너 안 /var/lib/postgresql에 저장돼요

```
~ $ docker exec pg ls /var/lib/postgresql
18
~ $ docker exec pg ls /var/lib/postgresql/18/docker
/var/lib/postgresql/18/docker:  base  global  pg_commit_ts  pg_dynshmem
pg_hba.conf  pg_ident.conf  pg_logical  pg_multixact  pg_notify
pg_replslot  pg_serial  pg_snapshots  pg_stat  pg_stat_tmp
pg_subtrans  pg_tblspc  pg_twophase  PG_VERSION  pg_wal
pg_xact  postgresql.auto.conf  postgresql.conf  postmaster.opts  postmaster.pid
```

버전 번호 폴더

1파트 env의 PGDATA=/var/lib/postgresql/18/docker가 이 폴더예요. 데이터베이스와 표가 이 안에 파일로 있어요.



> 18부터 자리가 바뀌었어요. 17까지는 /var/lib/postgresql/data였어요. 인터넷의 예전 글을 따라 할 때 버전을 같이 봐요.

4. 볼륨과 compose

-v pg-data:/var/lib/postgresql을 붙이면 지웠다 켜도 남아요

```
~ $ docker run -d --name pg -e POSTGRES_PASSWORD=cafe1234 … -v pg-data:/var/lib/postgresql postgres:18
fd1395c21f5b…
~ $ docker exec -it pg psql -U postgres
postgres=# CREATE DATABASE shop;
CREATE DATABASE
postgres=# \c shop
You are now connected to database "shop" as user "postgres".
shop=# CREATE TABLE orders (…);
CREATE TABLE
shop=# INSERT INTO orders VALUES (1, 'latte', 2);
INSERT 0 1
shop=# INSERT INTO orders VALUES (2, 'americano', 1);
INSERT 0 1
~ $ docker stop pg
pg
~ $ docker rm pg
pg
~ $ docker volume ls
DRIVER    VOLUME NAME
local     pg-data
```

볼륨은 남아 있어요

```
~ $ docker run -d --name pg … postgres:18
8869b886033d…
~ $ docker logs pg
PostgreSQL Database directory appears to contain a
database; Skipping initialization
…
~ $ docker exec -it pg psql -U postgres -d shop
shop=# SELECT * FROM orders;
 id |   menu    | qty
----+-----------+-----
  1 | latte     |   2
  2 | americano |   1
(2 rows)
```

새 컨테이너지만 pg-data의 데이터를 그대로 읽었어요.

4. 볼륨과 compose

길어진 docker run을 compose.yaml에 옮겨 적어요

~/cafe-pg/compose.yaml

```yaml
services:
  pg:
    image: postgres:18
    environment:
      POSTGRES_PASSWORD: cafe1234
      POSTGRES_DB: cafe
    ports:
      - "5432:5432"
    volumes:
      - pg-data:/var/lib/postgresql

volumes:
  pg-data:
```

- --name pg 대신 서비스 이름
- postgres:18
- -e POSTGRES_PASSWORD=cafe1234
- -e POSTGRES_DB=cafe
- -p 5432:5432
- -v pg-data:/var/lib/postgresql
- 이름 있는 볼륨은 여기 한 번 더

4. 볼륨과 compose

docker compose up -d로 켜고, 서비스 이름 pg로 들어가요

```
~/cafe-pg $ docker compose up -d
[+] up 3/3
 ✔ Network cafe-pg_default Created                                    0.0s
 ✔ Volume cafe-pg_pg-data  Created                                    0.0s
 ✔ Container cafe-pg-pg-1  Started                                    0.2s

~/cafe-pg $ docker compose ps
NAME           IMAGE         COMMAND                  SERVICE   CREATED         STATUS         PORTS
cafe-pg-pg-1   postgres:18   "docker-entrypoint.s…"   pg        3 seconds ago   Up 2 seconds   0.0.0.0:5432->5432/tcp
```

폴더 이름이 앞에 붙어요

```
~/cafe-pg $ docker compose exec pg psql -U postgres -d cafe
cafe=# CREATE TABLE orders (…);
CREATE TABLE
cafe=# INSERT INTO orders VALUES (1, 'latte', 2);
INSERT 0 1
cafe=# INSERT INTO orders VALUES (2, 'americano', 1);
INSERT 0 1
cafe=# SELECT * FROM orders;
 id |   menu    | qty
----+-----------+-----
  1 | latte     |   2
  2 | americano |   1
(2 rows)
```

서비스 이름 pg, POSTGRES_DB로 만든 cafe

5. 자주 나는 오류

비밀번호 -e를 빠뜨리면 켜지자마자 꺼져요

```
~ $ docker run -d --name pg postgres:18
37d52c826c5d…

~ $ docker ps -a
CONTAINER ID   IMAGE         COMMAND                  CREATED         STATUS                     PORTS     NAMES
37d52c826c5d   postgres:18   "docker-entrypoint.s…"   3 seconds ago   Exited (1) 2 seconds ago             pg

~ $ docker logs pg
Error: Database is uninitialized and superuser password is not specified.
       You must specify POSTGRES_PASSWORD to a non-empty value for the
       superuser. For example, "-e POSTGRES_PASSWORD=password" on "docker run".
…
```

이름 pg가 남아 있어요. docker rm pg로 지우고 -e POSTGRES_PASSWORD를 붙여 다시 켜요.

5. 자주 나는 오류

켜자마자 들어가면 아직 준비 중이라 접속이 안 돼요

```
~ $ docker run -d --name pg -e POSTGRES_PASSWORD=cafe1234 -p 5432:5432 postgres:18
b0771208641d…
~ $ docker exec -it pg psql -U postgres
psql: error: connection to server on socket "/var/run/postgresql/.s.PGSQL.5432" failed: No such file
or directory
Is the server running locally and accepting connections on that socket?
```

```
~ $ docker logs pg
…
PostgreSQL init process complete; ready for start up.
…
… LOG:  database system is ready to accept connections
```

이 줄이 보이면 다시 들어가요

5. 자주 나는 오류

5432번을 이미 쓰고 있으면 port is already allocated가 나요

```
~ $ docker run -d --name pg3 -e POSTGRES_PASSWORD=cafe1234 -p 5432:5432 postgres:18
e18a2c2ced77…
docker: Error response from daemon: failed to set up container networking: driver failed programming external
connectivity on endpoint pg3 (aefa57cd7ba24e64d17bfc3b31c0ae4eb0b7713a9145ae62afc4c453e74eeb37): Bind for
0.0.0.0:5432 failed: port is already allocated

Run 'docker run --help' for more information

~ $ docker ps -a
CONTAINER ID   IMAGE         COMMAND                  CREATED                  STATUS          PORTS                    NAMES
e18a2c2ced77   postgres:18   "docker-entrypoint.s…"   Less than a second ago   Created                                  pg3
4087ba29a61a   postgres:18   "docker-entrypoint.s…"   5 seconds ago            Up 4 seconds    0.0.0.0:5433->5432/tcp   pg2
b0771208641d   postgres:18   "docker-entrypoint.s…"   44 seconds ago           Up 43 seconds   0.0.0.0:5432->5432/tcp   pg
```

앞의 pg가 5432번을 쓰는 중이에요. 하나를 끄거나, 내 컴퓨터 쪽 번호만 -p 5433:5432로 바꿔요.

5. 자주 나는 오류

psql에서 자주 하는 실수 두 가지예요

1. 세미콜론을 빠뜨렸어요

```
shop=# SELECT * FROM orders
shop-# ;
 id |   menu    | qty
----+-----------+-----
  1 | latte     |   2
  2 | americano |   1
(2 rows)
```

=#가 -#로 바뀌면 문장이 아직 안 끝났다는 뜻이에요. ;만 치고 Enter를 눌러요.

2. 데이터베이스 이름을 잘못 썼어요

```
postgres=# \c shopp
connection to server on socket
"/var/run/postgresql/.s.PGSQL.5432" failed:
FATAL:  database "shopp" does not exist
Previous connection kept
```

없는 이름이라 못 옮기고, 원래 있던 postgres에 그대로 남아요. \l로 이름을 보고 다시 쳐요.

5. 자주 나는 오류

예전 글처럼 /var/lib/postgresql/data에 붙이면 18은 켜지지 않아요

```
~ $ docker run -d --name pg -e POSTGRES_PASSWORD=cafe1234 -p 5432:5432 -v old-data:/var/lib/postgresql/data postgres:18
e9a6d4089ae5…
~ $ docker ps -a
CONTAINER ID   IMAGE          COMMAND                  CREATED         STATUS                     PORTS     NAMES
e9a6d4089ae5   postgres:18    "docker-entrypoint.s…"   4 seconds ago   Exited (1) 3 seconds ago             pg
```

```
~ $ docker logs pg
Error: in 18+, these Docker images are configured to store database data in a
…
         /var/lib/postgresql/data (unused mount/volume)
…
       The suggested container configuration for 18+ is to place a single mount
       at /var/lib/postgresql which will then place PostgreSQL data in a
```

17까지 쓰던 자리

~~**/var/lib/postgresql/data**~~

18부터 붙이는 자리

/var/lib/postgresql

docker rm pg로 지우고
`-v pg-data:/var/lib/postgresql`로
다시 켜요.

5. 자주 나는 오류

볼륨에 데이터가 있으면 바꾼 비밀번호는 무시돼요

```
~ $ docker stop pg
pg
~ $ docker rm pg
pg
~ $ docker run -d --name pg -e POSTGRES_PASSWORD=new5678 …
294aa7bcf3b0…
~ $ docker logs pg
PostgreSQL Database directory appears to contain a database; Skipping
initialization
…
```

볼륨은 그대로

```
~ $ docker run … psql -h host.docker.internal -U postgres
Password for user postgres:
psql: error: connection to server at "host.docker.internal"
(172.17.0.1), port 5432 failed: FATAL:  password
authentication failed for user "postgres"
```

new5678

```
~ $ docker run … -U postgres -d shop
Password for user postgres:
shop=# SELECT * FROM orders;
 id |   menu    | qty
----+-----------+-----
  1 | latte     |   2
  2 | americano |   1
(2 rows)
```

처음 비밀번호 cafe1234

POSTGRES_로 시작하는 -e는 데이터 폴더가 비어 있을 때 한 번만 읽어요.

처음부터 다시 하려면 docker volume rm pg-data로 볼륨까지 지워요. 넣어 둔 데이터도 같이 지워져요.

6. 실제 상황 예제

compose down 하고 다음 날 up 해도 어제 주문이 남아 있어요

1. 문 닫을 때

```
~/cafe-pg $ docker compose down
[+] down 2/2
 ✔ Container cafe-pg-pg-1  Removed   0.2s
 ✔ Network cafe-pg_default Removed   0.2s
~/cafe-pg $ docker volume ls
DRIVER    VOLUME NAME
local     cafe-pg_pg-data
```

볼륨은 남아요

2. 다음 날 문 열 때

```
~/cafe-pg $ docker compose up -d
[+] up 2/2
 ✔ Network cafe-pg_default Created   0.0s
 ✔ Container cafe-pg-pg-1  Started   0.2s
~/cafe-pg $ docker compose exec pg psql -U postgres -d cafe
cafe=# INSERT INTO orders VALUES (3, 'cake', 1);
INSERT 0 1
cafe=# SELECT * FROM orders;
 id |   menu    | qty
----+-----------+-----
  1 | latte     |   2
  2 | americano |   1
  3 | cake      |   1
(3 rows)
```

down은 컨테이너와 네트워크만 지워요. 볼륨을 지우는 down -v는 연습을 끝낼 때만 쳐요.

6. 실제 상황 예제

init 폴더에 SQL 파일을 두면 처음 켤 때 메뉴가 들어가요

~/cafe-pg/init/menu.sql

```
1  CREATE TABLE menu (id INT PRIMARY KEY, name VARCHAR(20), price INT);
2  INSERT INTO menu VALUES (1, 'americano', 4000);
3  INSERT INTO menu VALUES (2, 'latte', 4500);
4  INSERT INTO menu VALUES (3, 'cake', 5000);
```

~/cafe-pg/compose.yaml (volumes 부분)

```
9      volumes:
10       - pg-data:/var/lib/postgresql
11       - ./init:/docker-entrypoint-initdb.d
```

```
~/cafe-pg $ docker compose up -d
…
~/cafe-pg $ docker compose logs pg
…
pg-1  | … running /docker-entrypoint-initdb.d/menu.sql
pg-1  | CREATE TABLE
pg-1  | INSERT 0 1
pg-1  | INSERT 0 1
```

```
~/cafe-pg $ docker compose exec pg psql -U postgres -d cafe
cafe=# \dt
          List of tables
 Schema | Name | Type  |  Owner
--------+------+-------+----------
 public | menu | table | postgres
(1 row)
cafe=# SELECT * FROM menu;
 id |   name    | price
----+-----------+-------
  1 | americano |  4000
  2 | latte     |  4500
  3 | cake      |  5000
(3 rows)
```

볼륨이 비어 있을 때만 실행해요. 다시 넣으려면 down -v 뒤에 up -d예요.

정리

33 MySQL과 컨테이너 쪽에서 다른 점이에요

	33 MySQL	34 PostgreSQL
이미지	`mysql:8.4`	`postgres:18`
관리자	`root`	`postgres`
관리자 비밀번호	`MYSQL_ROOT_PASSWORD`	`POSTGRES_PASSWORD`
처음 만들 DB	`MYSQL_DATABASE`	`POSTGRES_DB`
앱 계정 -e	`MYSQL_USER` 일반 계정을 하나 더	`POSTGRES_USER` 관리자 이름이 바뀌어요
포트	`3306`	`5432`
데이터 자리	`/var/lib/mysql`	`/var/lib/postgresql`
들어가기	`docker exec -it db mysql -u root -p`	`docker exec -it pg psql -U postgres`
안에서 비밀번호	물어요	안 물어요

정리

SQL은 같고, 목록을 보는 명령만 달라요

둘 다 똑같이 쳐요

```
CREATE DATABASE shop;
CREATE TABLE orders (…);
INSERT INTO orders VALUES (…);
SELECT * FROM orders;
DROP TABLE orders;
DROP DATABASE shop;
```

문장 끝 ;도 같아요.

	MySQL	PostgreSQL
데이터베이스 목록	`SHOW DATABASES;`	`\l`
다른 데이터베이스로	`USE shop;`	`\c shop`
표 목록	`SHOW TABLES;`	`\dt`
표 모양	`DESCRIBE orders;`	`\d orders`
나가기	`exit`	`\q`
한 문장만 실행	`mysql … -e "…"`	`psql … -c "…"`
덜 끝난 문장 표시	`->`	`-#`

PostgreSQL의 목록 명령은 \로 시작하고 ;가 없어요.

정리

막히면 이렇게 풀어요

이런 글이 보이면	이렇게 해요
`connection to server on socket … failed` 켜는 중이라 아직 서버가 없어요	**logs에서 ready 보고 다시**
`password authentication failed` 비밀번호가 틀렸거나 볼륨의 처음 비밀번호예요	**처음 정한 비밀번호로**
`Exited (1), POSTGRES_PASSWORD` -e POSTGRES_PASSWORD가 빠졌어요	`docker rm pg` 뒤 `-e` 붙이기
`Error: in 18+` 17까지 쓰던 /var/lib/postgresql/data에 붙였어요	`-v pg-data:/var/lib/postgresql`
`port is already allocated` 5432번을 다른 것이 쓰고 있어요	**하나 끄거나 -p 5433:5432**
`permission denied to create database` 일반 계정은 데이터베이스를 못 만들어요	`-U postgres`로 들어가기
표시가 -#로 바뀜 문장 끝 ;가 빠졌어요	**;만 치고 Enter**

무엇이든 막히면 docker ps -a로 켜져 있는지 보고, docker logs pg를 먼저 읽어요.

정리

한 장 요약

켜고 들어가기

`docker run -d --name pg -e POSTGRES_PASSWORD=… -p 5432:5432`

관리자 비밀번호를 주고 켜기

`docker logs pg`

ready 두 번 보기

`docker exec -it pg psql -U postgres`

관리자로 들어가기

`-d shop / -c "…"`

데이터베이스 고르기 / 한 문장만

psql 안에서

`\l \c shop`

목록 보기 / 옮기기

`\dt \d orders`

표 목록 / 표 모양

`\du`

계정 목록

`\q`

나가기

데이터 지키기

`-v pg-data:/var/lib/postgresql`

18부터 이 자리

`docker compose up -d / down`

볼륨은 남아요

`docker compose down -v`

볼륨까지 지우기

`./init:/docker-entrypoint-initdb.d`

처음 켤 때 SQL 실행

SQL은 MySQL과 같아요. 목록 명령과 계정, 데이터 자리만 챙기면 돼요.

나중에는 파이썬 코드에서 이 데이터베이스에 붙어 주문을 넣고 꺼내 봐요.